WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR RESTAURANT TABLE RESERVATION AND FOOD SERVICE MANAGEMENT SYSTEM

Program	B.Tech in Computer Science and Engineering (AI & ML)
Course & Code	Database Management Systems (DBMS)
Section	AIML Rhinos
Batch	2025 - 2029
Course Faculty	Prof. Sahithi, School of Technology
Date of Submission	October 2026
Academic Year	2026 - 2027

Submitted by:

Roll Number	Student Name
25WU0102107	J. Sanjith

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for giving me the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I express my heartfelt thanks to my course faculty, Prof. Sahithi, for continuous guidance, constructive suggestions and support throughout the design and development of this project, which was presented in three progressive reviews. The guidance helped me understand the practical use of core relational database concepts, including Entity-Relationship (ER) modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), SQL constraint definition, stored procedures, triggers, transaction management and concurrency control.

This PBL activity has strengthened my understanding of database design and transactional integrity, and gave me hands-on experience in building a practical solution for the day-to-day operations of a restaurant.

UNDERTAKING

I hereby declare that the project entitled "DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR RESTAURANT TABLE RESERVATION AND FOOD SERVICE MANAGEMENT SYSTEM", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of my course faculty, Prof. Sahithi.

I undertake that the project report, database design, SQL scripts, application source code, results and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date: October 15, 2026

Place: Hyderabad, Telangana

Signature:

Name: J. Sanjith

Program: B.Tech CSE (AI & ML) - AIML Rhinos

Roll Number: 25WU0102340

Academic Year: 2026-2027, School of Technology, Woxsen University

TABLE OF CONTENTS

- Abstract & Keywords
- List of Tables
- List of Figures
- List of Acronyms
- 1. Problem Statement, Solution Strategy and ER Diagram
  - 1.1 Problem Statement
  - 1.2 Objectives
  - 1.3 Scope and Assumptions
  - 1.4 Proposed Solution Strategy
  - 1.5 Functional Requirements
  - 1.6 Entity-Relationship Diagram (ERD)
  - 1.7 Entity and Relationship Summary
- 2. Logical Database Design
  - 2.1 Relational Schema
  - 2.2 Keys and Integrity Constraints
  - 2.3 Functional Dependencies
  - 2.4 Normalization up to Third Normal Form (3NF)
  - 2.5 Data Dictionary
  - 2.6 Implementation of Logical Design: DDL, Procedures and Triggers
- 3. Application Development Using Logical Database Design
  - 3.1 Application Overview
  - 3.2 Development Environment and Database Connectivity
  - 3.3 Application Modules and Database Mapping
  - 3.4 CRUD Operations
  - 3.5 Business Rules, Validation and Error Handling
  - 3.6 Reports and Database Operations

    - 3.7 Application Screens
    - 3.8 Testing and Results
- 4. Conclusion
    - 4.1 Limitations and Future Enhancements
- 5. References
- 6. Appendices
    - Appendix A - Database Script (Key Objects)
    - Appendix B - Application Source Code (Core Functions)
    - Appendix C - Demonstration Output and Test Data

LIST OF TABLES

Table	Title
Table 1	Scope and Assumptions
Table 2	Proposed Solution Strategy
Table 3	Functional Requirements
Table 4	Entity and Relationship Summary
Table 5	Relational Schema
Table 6	Keys and Integrity Constraints
Table 7	Normalization Status up to 3NF
Table 8	Comprehensive Data Dictionary
Table 9	Database Implementation Environment
Table 10	Application Development Environment
Table 11	Application Module-to-Database Mapping
Table 12	CRUD Operations
Table 13	Business Rules, Validation and Error Handling
Table 14	Reports and Database Operations
Table 15	Test Cases and Verification Outcomes

Table	Title
Table 16	Evolution of the Project across the Three Reviews

LIST OF FIGURES

Figure	Title
Figure 1	Entity-Relationship Diagram of the Proposed System
Figure 2	System Architecture of the Application
Figure 3	Reservation, Ordering and Billing Output
Figure 4	Operational Reports Output
Figure 5	Top Dishes by Orders (Sample Data)

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram

Acronym	Expansion
FD	Functional Dependency
FK	Foreign Key
GST	Goods and Services Tax
PK	Primary Key
QR	Quick Response (code)
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
SQL	Structured Query Language
UPI	Unified Payments Interface

ABSTRACT

A busy restaurant handles a continuous flow of guests, tables, reservations, orders, kitchen tickets and payments. When these are managed in a notebook, on paper slips and with hand-added bills, tables are double-booked, orders are mixed up between the waiter and the kitchen, bills are miscalculated, no-shows go unnoticed, and the owner has no stored data from which to study popular dishes, busy hours or daily revenue. The objective of this project is to design, normalize and implement a Relational Database Management System (RDBMS) that automates table reservation, order taking, kitchen flow, billing and reporting for a restaurant.

The design evolved over three reviews: a five-table ER design with a basic reservation, order and bill total (Review 1), a ten-table schema normalized to 3NF with slot-based reservations, best-fit table allocation, billing with GST and service charge, stored procedures, triggers and test cases (Review 2), and a final sixteen-table system with a waiting list, coupons, ingredient inventory, a kitchen status flow, feedback, role-based security, an audit log, concurrency control, indexes, views and reports (Review 3). Critical business rules, namely no overlapping reservation for a table, a valid reservation slot, a positive dish quantity, one line per dish in an order, a bill derived from stored prices and one payment per bill, are enforced through declarative constraints, stored procedures and triggers. The MySQL 8 script defines the complete database, and the application layer

calls the same stored procedures so that every client receives identical behaviour. A suite of test cases covering reservation, ordering, billing, inventory, security and audit scenarios is defined to verify the rules.

KEYWORDS

Relational Database, ER Diagram, Normalization, SQL, MySQL, Restaurant Management, Table Reservation, Billing, Triggers, Concurrency Control, Integrity Constraints

CHAPTER 1: PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Restaurants of every size run many activities in parallel: guests phone or walk in to reserve tables, parties are seated, dishes are ordered and prepared, bills are settled and tables are cleared for the next guests. In many restaurants this information still lives in a booking notebook, on handwritten order slips and in hand-added bills. Nothing links a guest to the reservation, the reservation to the dishes ordered, or the order to the money collected.

This fragmentation causes several recurring problems. Reservations written in a register lead to clashes, with two parties promised the same table, and to unnoticed no-shows. Table status is tracked by sight, so a large table may be given to a small party while a party of six waits. Handwritten slips get lost or misread between the waiter and the kitchen, and the chef cannot see which orders are waiting. Bills added up by hand contain arithmetic mistakes and tax errors, and past prices cannot be verified. Ingredient stock is checked only when something runs out. Without stored records, owners cannot see popular dishes, busy hours or daily revenue. There is therefore a clear need for a centralized, normalized relational database that acts as one reliable source of truth for tables, reservations, orders and bills, enforces integrity rules, allocates tables safely and automates billing.

1.2 Objectives

The primary objectives of this project are:

1. Design a conceptual Entity-Relationship (ER) model covering the sixteen core entities of a restaurant, with precise cardinalities.

2. Derive the relational schema and normalize it to Third Normal Form (3NF) using functional dependencies, removing insertion, deletion and update anomalies.
3. Implement DDL constraints, stored procedures and triggers that enforce the key business rules: slot-based reservation with no overlap, best-fit table allocation, one line per dish in an order, a valid kitchen status flow and one payment per bill.
4. Implement a billing engine that computes the subtotal from stored dish prices, applies a coupon discount, GST and service charge, and records the payment.
5. Prevent double booking under concurrent requests by using transactions and row locking, and extend the system with a waiting list, ingredient inventory, feedback, role-based access control and an audit log.
6. Produce SQL views, indexes and reports for free tables, the kitchen queue, daily revenue, top-selling dishes, peak reservation hours and low-stock ingredients, and build an application that supports the complete workflow from guest registration to payment.

1.3 Scope and Assumptions

The scope boundaries and technical assumptions of this implementation are summarized in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	Customer, table and menu master data; slot-based table reservation with overlap check and best-fit allocation; waiting list; orders with a kitchen status flow; billing with GST, service charge and coupons; payment records; ingredient inventory with recipe-based stock deduction; guest feedback; staff roles; audit log; views and reports.
Out of Scope / Exclusions	Live online payment gateway (UPI integration); mobile app with online booking; QR-code menu and table ordering; SMS or email notifications; multi-branch deployment; delivery and take-away orders; merging tables for very large parties.

Item	Description
Assumptions	A single restaurant branch is modelled. One reservation holds exactly one table. The reservation slot length is chosen at booking time. GST and service charge default to 5% each and are stored on every bill. All prices are in Indian rupees. MySQL 8 with the InnoDB engine is the target database.

1.4 Proposed Solution Strategy

The solution combines a normalized relational design with business rules placed inside the database itself, as summarized in Table 2.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Approach
Primary users / roles	Guest (reservation), waiter (orders and bills), chef (kitchen queue), manager (menu, prices, staff and reports).
Modules	Table and reservation management; menu and ordering; kitchen and inventory; billing and payments; security, audit and reports.
Proposed workflow	Reservation: (1) guest details recorded, (2) free table of the right size found, (3) reservation saved and the table held, (4) guest arrives and the table becomes OCCUPIED. Order and bill: (5) dishes ordered for the table, (6) order rows stored with quantity, (7) bill total computed from stored prices, (8) table marked AVAILABLE after payment.
Database approach	Sixteen tables in 3NF with primary keys, foreign keys, CHECK, UNIQUE and ENUM constraints; stored procedures, a function and triggers hold the business rules.

Solution Element	Project-Specific Approach
Concurrency	Table allocation runs inside one transaction that locks the chosen table row with SELECT … FOR UPDATE; a UNIQUE key on table and slot is the safety net.
Important business rules	No overlapping reservation per table; slot end later than slot start; quantity above zero; one line per dish per order; bill derived from stored prices; one payment per bill; kitchen status moves PLACED, PREPARING, READY, SERVED in order.
Expected reports	Free tables, kitchen queue, daily revenue, top-selling dishes, peak reservation hours, low-stock ingredients.

1.5 Functional Requirements

The functional requirements of the system are listed and prioritized in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall store guest details and find a guest by phone number.	High / Core Master Data
FR-02	The system shall maintain dining tables with capacity and live status (AVAILABLE or OCCUPIED).	High / Core Master Data
FR-03	The system shall reserve a table for a date, time slot and party size and shall reject any overlapping slot on the same table.	High / Integrity Rule
FR-04	The system shall allocate the smallest table whose capacity is at least the party size and that has no overlapping reservation.	High / Core Workflow
FR-05	The system shall add a walk-in guest to a waiting list when no table is free.	Medium / Advanced

Req. ID	Functional Requirement	Remarks / Priority
FR-06	The system shall maintain menu categories and dishes with price and availability.	High / Core Master Data
FR-07	The system shall record an order with one row per dish and quantity, and move it through PLACED, PREPARING, READY and SERVED.	High / Core Workflow
FR-08	The system shall compute the bill from the stored prices, apply an optional coupon, then add GST and service charge.	High / Billing
FR-09	The system shall record one payment per bill, mark the bill PAID and release the table.	High / Billing
FR-10	The system shall reduce ingredient stock from the recipe of each dish ordered and list low-stock ingredients.	Medium / Inventory
FR-11	The system shall store guest ratings for dishes and service after payment.	Low / Feedback
FR-12	The system shall restrict each staff role to the data it needs and shall log changes to bills, payments and menu prices.	High / Security
FR-13	The system shall provide reports for top dishes, daily revenue, peak reservation hours and low stock.	Medium / Analytics

1.6 Entity-Relationship Diagram (ERD)

The conceptual Entity-Relationship model is shown in Figure 1. It grew over three reviews. Review 1 modelled five entities (Customer, DiningTable, Reservation, MenuItem and FoodOrder). Review 2 split FoodOrder into Orders and OrderItem and added Staff, MenuCategory, Bill and Payment, giving ten entities. Review 3 added WaitList, Ingredient, RecipeItem, Feedback, Coupon and AuditLog, giving the final sixteen entities.

A customer makes many reservations, and a dining table can be reserved many times at different slots. A reservation can have many orders, and each order is taken by one staff member. An order contains many dish rows (OrderItem); each row refers to one menu

item, and each menu item belongs to one category. An order has one bill, a bill has one payment, and a bill may use one coupon. A menu item uses many ingredients through its recipe, and an ingredient can appear in many recipes.

Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 documents the catalog of entities and relationships with their keys and purpose.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
CUSTOMER	customer_id (PK), full_name, phone (UNIQUE), email	Guest contact details; one row per guest.
DINING_TABLE	table_id (PK), table_no (UNIQUE), capacity, status	Physical tables with seating capacity and live status.
RESERVATION	reservation_id (PK), customer_id (FK), table_id (FK), slot_start, slot_end, party_size, status	Booking of one table for a time slot.
WAIT_LIST	wait_id (PK), customer_id (FK), party_size, requested_at, status	Queue of walk-in guests when no table is free.
MENU_CATEGORY	category_id (PK), category_name (UNIQUE), display_order, is_active	Groups dishes such as Starter, Main, Beverage, Dessert.
MENU_ITEM	item_id (PK), category_id (FK), item_name, price, is_available	Dishes with price and availability.
INGREDIENT	ingredient_id (PK), ingredient_name, unit, stock_qty, reorder_level	Raw material stock.
RECIPE_ITEM	item_id (FK), ingredient_id (FK), qty_per_dish	Quantity of each ingredient used by one dish.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
ORDERS	order_id (PK), reservation_id (FK), staff_id (FK), status, order_time	Order header with kitchen status.
ORDER_ITEM	order_item_id (PK), order_id (FK), item_id (FK), quantity, unit_price	One row per dish ordered.
FEEDBACK	feedback_id (PK), reservation_id (FK), item_id (FK, optional), rating, comments	Guest rating of dishes and service.
BILL	bill_id (PK), order_id (FK, UNIQUE), coupon_id (FK, optional), subtotal, discount, gst, service_charge, total, status	Bill for one order.
PAYMENT	payment_id (PK), bill_id (FK, UNIQUE), amount, pay_mode, paid_at	One payment per bill.
COUPON	coupon_id (PK), code (UNIQUE), discount_pct, valid_from, valid_to, is_active	Percentage discount offer.
STAFF	staff_id (PK), username (UNIQUE), full_name, role, shift	Manager, waiter and chef accounts.
AUDIT_LOG	log_id (PK), table_name, record_id, action, old_value, new_value, changed_by, changed_at	Trail of changes to sensitive tables.
CUSTOMER 1 : N RESERVATION	One customer makes many reservations.	customer_id foreign key.
DINING_TABLE 1 : N RESERVATION	One table is reserved many times at different slots.	table_id foreign key; no overlap.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
RESERVATION 1 : N ORDERS	One reservation can have many orders.	reservation_id foreign key.
STAFF 1 : N ORDERS	One waiter takes many orders.	staff_id foreign key.
ORDERS 1 : N ORDER_ITEM	One order has many dish rows.	order_id foreign key.
MENU_ITEM 1 : N ORDER_ITEM	One dish appears in many order rows.	item_id foreign key.
MENU_CATEGORY 1 : N MENU_ITEM	One category holds many dishes.	category_id foreign key.
ORDERS 1 : 1 BILL	One order has one bill.	UNIQUE order_id.
BILL 1 : 1 PAYMENT	One bill is paid once.	UNIQUE bill_id.
COUPON 1 : N BILL	One coupon can be used on many bills.	coupon_id nullable foreign key.
MENU_ITEM M : N INGREDIENT	A dish uses many ingredients and vice versa.	Resolved by RECIPE_ITEM.
CUSTOMER 1 : N WAIT_LIST	One guest can wait more than once.	customer_id foreign key.
RESERVATION 1 : N FEEDBACK	One visit can receive several ratings.	reservation_id foreign key.

CHAPTER 2: LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model is mapped to sixteen relations. Primary keys and foreign keys are defined for every relation, and Table 5 lists the relations with their module group, which follows the grouping used in the final system architecture.

Table 5. Relational Schema

S.No	Table Name	Group	Purpose
1	Customer	Guests	Who dines: guest contact details.
2	DiningTable	Guests	Where guests dine: tables with capacity and status.
3	Reservation	Guests	Booking of a table for a date, time slot and party size.
4	WaitList	Guests	Walk-in guests queued when no table is free.
5	MenuCategory	Menu	Dish categories with display order.
6	MenuItem	Menu	Dishes with category, price and availability.
7	Ingredient	Menu	Ingredient stock and reorder level.
8	RecipeItem	Menu	Ingredient quantities used by each dish.
9	Orders	Orders	Order header linked to a reservation and a waiter.
10	OrderItem	Orders	One row per dish ordered, with quantity and price.
11	Feedback	Orders	Ratings given by guests.
12	Bill	Billing	Subtotal, discount, GST, service charge and total.
13	Payment	Billing	Payment mode, amount and time.
14	Coupon	Billing	Discount offers with validity period.
15	Staff	Platform	Staff accounts with role and shift.
16	AuditLog	Platform	Record of who changed what.

2.2 Keys and Integrity Constraints

Declarative and procedural constraints guarantee consistency and prevent illegal data states. Table 6 maps the constraints to the business rules they enforce.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Customer	PK, UNIQUE	customer_id; phone	One row per guest; a phone number identifies one guest.
DiningTable	PK, UNIQUE, CHECK	table_id; table_no; capacity > 0	Unique table numbers; capacity is positive.
Reservation	PK, FK, CHECK, UNIQUE	reservation_id; customer_id, table_id; slot_end > slot_start; (table_id, slot_start)	Valid slot; guest and table must exist; no two reservations start together on one table.
WaitList	PK, FK, CHECK	wait_id; customer_id; party_size > 0	Only registered guests can wait.
MenuCategory	PK, UNIQUE	category_id; category_name	No duplicate categories.
MenuItem	PK, FK, CHECK	item_id; category_id; price >= 0	Every dish belongs to a category; price cannot be negative.
Ingredient	PK, UNIQUE, CHECK	ingredient_id; ingredient_name; stock_qty >= 0	Stock can never become negative.
RecipeItem	Composite PK, FK	(item_id, ingredient_id)	One recipe line per dish and ingredient.
Orders	PK, FK, ENUM, trigger	order_id; reservation_id, staff_id; status	Order tied to a reservation and a waiter; status moves in the allowed order.
OrderItem	PK, FK, CHECK, UNIQUE	order_item_id; quantity > 0; (order_id, item_id)	Positive quantity; one line per dish per order.
Feedback	PK, FK, CHECK	feedback_id; rating between 1 and 5	Valid rating linked to a visit.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Bill	PK, FK, UNIQUE	bill_id; order_id	One bill per order.
Payment	PK, FK, UNIQUE	payment_id; bill_id	One payment per bill.
Coupon	PK, UNIQUE, CHECK	coupon_id; code; discount_pct between 0 and 100	Unique code and valid percentage.
Staff	PK, UNIQUE, ENUM	staff_id; username; role	Unique login; role limited to MANAGER, WAITER or CHEF.
AuditLog	PK	log_id	Every logged change has its own identity.

2.3 Functional Dependencies

The non-trivial functional dependencies for each relation, which drive the normalization in Section 2.4, are:

1. Customer: customer_id → full_name, phone, email; phone → customer_id (phone is a candidate key).
2. DiningTable: table_id → table_no, capacity, status; table_no → table_id.
3. Reservation: reservation_id → customer_id, table_id, slot_start, slot_end, party_size, status; (table_id, slot_start) → reservation_id.
4. WaitList: wait_id → customer_id, party_size, requested_at, status.
5. MenuCategory: category_id → category_name, display_order, is_active; category_name → category_id.
6. MenuItem: item_id → category_id, item_name, price, is_available.
7. Ingredient: ingredient_id → ingredient_name, unit, stock_qty, reorder_level.
8. RecipeItem: (item_id, ingredient_id) → qty_per_dish.
9. Staff: staff_id → username, full_name, role, shift; username → staff_id.
10. Orders: order_id → reservation_id, staff_id, status, order_time.

11. OrderItem: order_item_id → order_id, item_id, quantity, unit_price; (order_id, item_id) → order_item_id.
12. Bill: bill_id → order_id, coupon_id, subtotal, discount, gst, service_charge, total, status; order_id → bill_id.
13. Payment: payment_id → bill_id, amount, pay_mode, paid_at; bill_id → payment_id.
14. Coupon: coupon_id → code, discount_pct, valid_from, valid_to, is_active; code → coupon_id.
15. Feedback: feedback_id → reservation_id, item_id, rating, comments.
16. AuditLog: log_id → table_name, record_id, action, old_value, new_value, changed_by, changed_at.

2.4 Normalization up to Third Normal Form (3NF)

Every relation was derived from a single wide, unnormalized order table, which is where the update anomalies of a manual system appear:

Order(order_id, customer_name, phone, table_no, capacity, waiter, item_name, category, price, quantity, tax, total)

Customer, table and dish details repeat on every row, so changing a phone number or a dish price means editing many rows (update anomaly), a table cannot be stored until an order exists (insertion anomaly), and deleting the last order of a dish deletes the dish (deletion anomaly).

- **1NF:** All attribute values are atomic and each row represents one dish ordered, so one order with three dishes becomes three rows in OrderItem. No repeating groups remain.
- **2NF:** Partial dependencies on part of the key are removed. Dish, table and customer details move to MenuItem, DiningTable and Customer, because they depend on item_id, table_id and customer_id alone and not on the whole order line.
- **3NF:** Transitive dependencies are removed. Category details move to MenuCategory (item_id → category_id → category_name) and waiter details move to Staff (order_id → staff_id → role, shift). Bill amounts are separated from the order, and payments are separated from the bill.
- **BCNF check:** In every relation each determinant is a candidate key (for example phone in Customer, code in Coupon and username in Staff are candidate keys), so the schema also satisfies Boyce-Codd Normal Form.

Table 7 summarizes the normalization status of all relations.

Table 7. Normalization Status up to 3NF

Relation	1NF	2NF	3NF	Remarks / Decomposition Basis
Customer	Yes	Yes	Yes	Single-attribute key; contact details only.
DiningTable	Yes	Yes	Yes	Table facts depend only on table_id.
Reservation	Yes	Yes	Yes	Customer and table details referenced by key.
WaitList	Yes	Yes	Yes	Guest referenced by customer_id.
MenuCategory	Yes	Yes	Yes	Category facts separated from dishes.
MenuItem	Yes	Yes	Yes	Category referenced by category_id.
Ingredient	Yes	Yes	Yes	Stock facts depend on ingredient_id.
RecipeItem	Yes	Yes	Yes	Resolves the dish and ingredient many-to-many link.
Orders	Yes	Yes	Yes	Waiter referenced by staff_id; dishes moved to OrderItem.
OrderItem	Yes	Yes	Yes	One row per dish; price captured at order time.
Feedback	Yes	Yes	Yes	Rating depends only on feedback_id.
Bill	Yes	Yes	Yes	Amounts separated from order and payment.
Payment	Yes	Yes	Yes	One payment per bill.
Coupon	Yes	Yes	Yes	Offer facts depend only on coupon_id.
Staff	Yes	Yes	Yes	Role and shift depend on staff_id.
AuditLog	Yes	Yes	Yes	Append-only log rows.

2.5 Data Dictionary

Table 8 gives the data dictionary of all attributes as implemented in the database.

Table 8. Comprehensive Data Dictionary

Table	Attribute	Datatype	Key / Constraint	Description
Customer	customer_id	INT	PK, AUTO_INCREMENT	Unique guest identifier.
Customer	full_name	VARCHAR(80)	NOT NULL	Guest name.
Customer	phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number; identifies a guest.
Customer	email	VARCHAR(100)	NULL	Optional email address.
Customer	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Registration time.
DiningTable	table_id	INT	PK, AUTO_INCREMENT	Unique table identifier.
DiningTable	table_no	VARCHAR(10)	NOT NULL, UNIQUE	Table number shown to staff.
DiningTable	capacity	TINYINT	NOT NULL, CHECK > 0	Number of seats.
DiningTable	status	ENUM	DEFAULT 'AVAILABLE'	AVAILABLE or OCCUPIED.
Reservation	reservation_id	INT	PK, AUTO_INCREMENT	Unique reservation identifier.
Reservation	customer_id	INT	FK, NOT NULL	Guest who booked.
Reservation	table_id	INT	FK, NOT NULL	Table held for the slot.
Reservation	slot_start	DATETIME	NOT NULL	Start of the booked slot.
Reservation	slot_end	DATETIME	NOT NULL, CHECK > slot_start	End of the booked slot.
Reservation	party_size	TINYINT	NOT NULL, CHECK > 0	Number of guests.
Reservation	status	ENUM	DEFAULT 'BOOKED'	BOOKED, SEATED, DONE or CANCELLED.
Reservation	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of booking.
WaitList	wait_id	INT	PK, AUTO_INCREMENT	Unique waiting entry.

Table	Attribute	Datatype	Key / Constraint	Description
WaitList	customer_id	INT	FK, NOT NULL	Waiting guest.
WaitList	party_size	TINYINT	NOT NULL, CHECK > 0	Size of the waiting party.
WaitList	requested_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the guest joined the queue.
WaitList	status	ENUM	DEFAULT 'WAITING'	WAITING, SEATED or LEFT.
MenuCategory	category_id	INT	PK, AUTO_INCREMENT	Unique category identifier.
MenuCategory	category_name	VARCHAR(40)	NOT NULL, UNIQUE	Starter, Main, Beverage, Dessert.
MenuCategory	display_order	TINYINT	DEFAULT 0	Position on the menu.
MenuCategory	is_active	BOOLEAN	DEFAULT TRUE	Hides a category without deleting it.
MenuItem	item_id	INT	PK, AUTO_INCREMENT	Unique dish identifier.
MenuItem	category_id	INT	FK, NOT NULL	Category of the dish.
MenuItem	item_name	VARCHAR(80)	NOT NULL, UNIQUE	Dish name.
MenuItem	price	DECIMAL(8,2)	NOT NULL, CHECK >= 0	Current selling price.
MenuItem	is_available	BOOLEAN	DEFAULT TRUE	Whether the dish can be ordered.
Ingredient	ingredient_id	INT	PK, AUTO_INCREMENT	Unique ingredient identifier.
Ingredient	ingredient_name	VARCHAR(60)	NOT NULL, UNIQUE	Ingredient name.
Ingredient	unit	VARCHAR(10)	NOT NULL	Unit of measure (kg, l, pcs).
Ingredient	stock_qty	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Quantity in stock.
Ingredient	reorder_level	DECIMAL(10,2)	DEFAULT 0	Stock level that triggers a low-stock report.
RecipeItem	item_id	INT	PK (part), FK	Dish.

Table	Attribute	Datatype	Key / Constraint	Description
RecipeItem	ingredient_id	INT	PK (part), FK	Ingredient used.
RecipeItem	qty_per_dish	DECIMAL(8,3)	NOT NULL, CHECK > 0	Ingredient quantity for one dish.
Staff	staff_id	INT	PK, AUTO_INCREMENT	Unique staff identifier.
Staff	username	VARCHAR(30)	NOT NULL, UNIQUE	Login name.
Staff	full_name	VARCHAR(80)	NOT NULL	Staff member name.
Staff	role	ENUM	NOT NULL	MANAGER, WAITER or CHEF.
Staff	shift	ENUM	NOT NULL	MORNING, EVENING or NIGHT.
Orders	order_id	INT	PK, AUTO_INCREMENT	Unique order identifier.
Orders	reservation_id	INT	FK, NOT NULL	Reservation the order belongs to.
Orders	staff_id	INT	FK, NOT NULL	Waiter who took the order.
Orders	status	ENUM	DEFAULT 'PLACED'	PLACED, PREPARING, READY or SERVED.
Orders	order_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the order was placed.
OrderItem	order_item_id	INT	PK, AUTO_INCREMENT	Unique order line.
OrderItem	order_id	INT	FK, NOT NULL	Parent order.
OrderItem	item_id	INT	FK, NOT NULL	Dish ordered.
OrderItem	quantity	TINYINT	NOT NULL, CHECK > 0	Number of portions.
OrderItem	unit_price	DECIMAL(8,2)	NOT NULL	Dish price captured at order time.
Feedback	feedback_id	INT	PK, AUTO_INCREMENT	Unique feedback entry.
Feedback	reservation_id	INT	FK, NOT NULL	Visit being rated.

Table	Attribute	Datatype	Key / Constraint	Description
Feedback	item_id	INT	FK, NULL	Dish rated; NULL for service.
Feedback	rating	TINYINT	NOT NULL, CHECK 1 to 5	Star rating.
Feedback	comments	VARCHAR(255)	NULL	Optional remarks.
Coupon	coupon_id	INT	PK, AUTO_INCREMENT	Unique coupon identifier.
Coupon	code	VARCHAR(20)	NOT NULL, UNIQUE	Code typed at billing.
Coupon	discount_pct	DECIMAL(5,2)	NOT NULL, CHECK 0 to 100	Percentage discount.
Coupon	valid_from, valid_to	DATE	NOT NULL	Validity window.
Coupon	is_active	BOOLEAN	DEFAULT TRUE	Switches a coupon off.
Bill	bill_id	INT	PK, AUTO_INCREMENT	Unique bill identifier.
Bill	order_id	INT	FK, NOT NULL, UNIQUE	Order being billed.
Bill	coupon_id	INT	FK, NULL	Coupon applied, if any.
Bill	subtotal	DECIMAL(10,2)	NOT NULL	Sum of quantity times unit price.
Bill	discount	DECIMAL(10,2)	DEFAULT 0	Coupon discount amount.
Bill	gst_rate, service_rate	DECIMAL(4,2)	DEFAULT 5.00	Rates in force when the bill was made.
Bill	gst, service_charge	DECIMAL(10,2)	NOT NULL	Tax and service amounts.
Bill	total	DECIMAL(10,2)	NOT NULL	Amount payable.
Bill	status	ENUM	DEFAULT 'UNPAID'	UNPAID or PAID.
Bill	generated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the bill was generated.
Payment	payment_id	INT	PK, AUTO_INCREMENT	Unique payment identifier.

Table	Attribute	Datatype	Key / Constraint	Description
Payment	bill_id	INT	FK, NOT NULL, UNIQUE	Bill that was paid.
Payment	amount	DECIMAL(10,2)	NOT NULL	Amount received.
Payment	pay_mode	ENUM	NOT NULL	CASH, CARD or UPI.
Payment	paid_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Payment time.
AuditLog	log_id	BIGINT	PK, AUTO_INCREMENT	Unique log entry.
AuditLog	table_name	VARCHAR(30)	NOT NULL	Table that changed.
AuditLog	record_id	INT	NOT NULL	Row that changed.
AuditLog	action	VARCHAR(20)	NOT NULL	INSERT or UPDATE.
AuditLog	old_value, new_value	VARCHAR(255)	NULL	Value before and after the change.
AuditLog	changed_by	VARCHAR(80)	NOT NULL	Database user that made the change.
AuditLog	changed_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of the change.

2.6 Implementation of Logical Design: DDL, Procedures and Triggers

The database implementation is summarized in Table 9. The complete script is reproduced in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS / Engine	MySQL 8 with the InnoDB engine, which provides ACID transactions, row-level locking, foreign keys and CHECK constraints.
Design tools	MySQL Workbench or draw.io for ER diagrams.

Item	Details
Database script	One SQL script containing the schema, procedures, function, triggers, views, indexes, roles and seed data.
Database objects	16 tables; 4 stored procedures (sp_reserve_table, sp_place_order, sp_generate_bill, sp_join_waitlist); 1 function (fn_bill_total); 6 triggers; 3 views; 3 indexes; 3 roles.
Business rules in the database	Constraints, procedures and triggers hold every rule, so any client, whether a form, a script or a future mobile app, receives the same behaviour.

Representative DDL. Two representative tables show how integrity is built into the schema.

```
CREATE TABLE Reservation (
  reservation_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL, table_id INT NOT NULL,
  slot_start DATETIME NOT NULL, slot_end DATETIME NOT NULL,
  party_size TINYINT NOT NULL CHECK (party_size > 0),
  status ENUM('BOOKED','SEATED','DONE','CANCELLED') DEFAULT 'BOOKED',
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  CHECK (slot_end > slot_start),
  UNIQUE (table_id, slot_start),
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
  FOREIGN KEY (table_id) REFERENCES DiningTable(table_id)
);
CREATE TABLE OrderItem (
  order_item_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT NOT NULL, item_id INT NOT NULL,
  quantity TINYINT NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(8,2) NOT NULL,
  UNIQUE (order_id, item_id),
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  FOREIGN KEY (item_id) REFERENCES MenuItem(item_id)
);
```

Smart table allocation. The procedure sp_reserve_table reads the party size and the requested slot, keeps the tables whose capacity is at least the party size, excludes tables that have an overlapping reservation, and picks the smallest fitting table. It then locks

that table row, re-checks for an overlap and only then inserts the reservation, all inside one transaction. If no table qualifies it returns NO TABLE FREE and saves nothing.

```
DELIMITER $$
CREATE PROCEDURE sp_reserve_table(
  IN p_customer INT, IN p_party TINYINT,
  IN p_start DATETIME, IN p_end DATETIME,
  OUT p_res_id INT, OUT p_msg VARCHAR(60))
BEGIN
  DECLARE v_table INT DEFAULT NULL;
  DECLARE v_lock INT;
  DECLARE v_clash INT DEFAULT 0;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK; SET p_res_id = NULL; SET p_msg = 'ERROR: rolled back';
  END;
  SET p_res_id = NULL;
  START TRANSACTION;
  SELECT t.table_id INTO v_table FROM DiningTable t
   WHERE t.capacity >= p_party
     AND NOT EXISTS (SELECT 1 FROM Reservation r
          WHERE r.table_id = t.table_id
            AND r.status IN ('BOOKED','SEATED')
            AND r.slot_start < p_end AND r.slot_end > p_start)
   ORDER BY t.capacity, t.table_id LIMIT 1;
  IF v_table IS NULL THEN
    SET p_msg = 'NO TABLE FREE'; ROLLBACK;
  ELSE
    SELECT table_id INTO v_lock FROM DiningTable
     WHERE table_id = v_table FOR UPDATE;
    SELECT COUNT(*) INTO v_clash FROM Reservation r
     WHERE r.table_id = v_table AND r.status IN ('BOOKED','SEATED')
       AND r.slot_start < p_end AND r.slot_end > p_start;
    IF v_clash > 0 THEN
      SET p_msg = 'NO TABLE FREE'; ROLLBACK;
    ELSE
      INSERT INTO Reservation (customer_id, table_id, slot_start, slot_end,
party_size)
      VALUES (p_customer, v_table, p_start, p_end, p_party);
      SET p_res_id = LAST_INSERT_ID(); SET p_msg = 'RESERVED'; COMMIT;
```

```
    END IF;
  END IF;
END$$
DELIMITER ;
```

Billing function and triggers. The function fn_bill_total applies the discount, GST and service charge to a subtotal. Two triggers keep the table status consistent: trg_order_after_insert sets the table to OCCUPIED when an order is placed, and trg_payment_after_insert marks the bill PAID, completes the reservation, frees the table and writes an audit entry.

```
CREATE FUNCTION fn_bill_total(p_subtotal DECIMAL(10,2), p_discount
DECIMAL(10,2),
                              p_gst DECIMAL(4,2), p_service DECIMAL(4,2))
RETURNS DECIMAL(10,2) DETERMINISTIC
RETURN ROUND((p_subtotal - p_discount) * (1 + (p_gst + p_service) / 100), 2);

DELIMITER $$
CREATE TRIGGER trg_order_after_insert AFTER INSERT ON Orders FOR EACH ROW
BEGIN
  UPDATE Reservation SET status = 'SEATED'
   WHERE reservation_id = NEW.reservation_id AND status = 'BOOKED';
  UPDATE DiningTable SET status = 'OCCUPIED'
   WHERE table_id = (SELECT table_id FROM Reservation
                     WHERE reservation_id = NEW.reservation_id);
END$$

CREATE TRIGGER trg_payment_after_insert AFTER INSERT ON Payment FOR EACH ROW
BEGIN
  DECLARE v_res INT;
  UPDATE Bill SET status = 'PAID' WHERE bill_id = NEW.bill_id;
  SELECT o.reservation_id INTO v_res FROM Bill b
    JOIN Orders o ON o.order_id = b.order_id WHERE b.bill_id = NEW.bill_id;
  UPDATE Reservation SET status = 'DONE' WHERE reservation_id = v_res;
  UPDATE DiningTable SET status = 'AVAILABLE'
   WHERE table_id = (SELECT table_id FROM Reservation WHERE reservation_id =
v_res);
  INSERT INTO AuditLog (table_name, record_id, action, new_value, changed_by)
  VALUES ('Payment', NEW.payment_id, 'INSERT', CONCAT('amount=', NEW.amount),
USER());
END$$
DELIMITER ;
```

CHAPTER 3: APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is a simple forms-based client for the restaurant. It does not contain business rules of its own. Instead every screen calls a stored procedure, a view or a plain query, so the rules live in the DBMS layer. Figure 2 shows the final architecture. Users, namely the guest, the waiter and chef, and the manager, work through three groups of screens: the reservation screen, the order and kitchen screen, and the billing and reports screen. These screens connect to the DBMS layer, which holds the sixteen tables in 3NF together with procedures, triggers, views, indexes and roles. Ingredient stock is updated automatically from every order through a trigger, so the application never edits stock directly.

Figure 2. System Architecture of the Application

3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming language	Python 3 for the simple booking and ordering forms (a Java client would work in the same way).
Database connectivity	MySQL Connector/Python opens a connection with the role of the logged-in staff member and calls procedures with CALL statements.
Database	MySQL 8 holds all restaurant data.
Design tools	MySQL Workbench or draw.io for ER diagrams.
Design principle	The design stays independent of the front end: all rules live in the database, so MySQL could be replaced by PostgreSQL without changing the logical design.

3.3 Application Modules and Database Mapping

Table 11 maps each application module to the database objects it uses.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables / Objects Used	Main Database Operations
Guest registration	Add or find a guest by phone.	Customer	INSERT; SELECT by phone.
Table management	Maintain tables and view live status.	DiningTable, vw_free_tables	SELECT; UPDATE status.
Reservation	Book a free table for a slot and party size.	Reservation, DiningTable, Customer	CALL sp_reserve_table.
Waiting list	Queue walk-in guests when full.	WaitList, Customer	CALL sp_join_waitlist; UPDATE status.

Module / Screen	Purpose	Tables / Objects Used	Main Database Operations
Menu management	Maintain categories, dishes and prices.	MenuCategory, MenuItem	INSERT, UPDATE; audit trigger on price.
Order taking	Record the dishes ordered for a table.	Orders, OrderItem, MenuItem	CALL sp_place_order.
Kitchen display	Chef sees open orders and updates status.	Orders, OrderItem, vw_kitchen_queue	SELECT view; UPDATE status.
Inventory	Stock reduced from recipes; low-stock list.	Ingredient, RecipeItem	Trigger on OrderItem; SELECT low stock.
Billing and payment	Generate bill, apply coupon, record payment.	Bill, Coupon, Payment	CALL sp_generate_bill; INSERT Payment.
Feedback	Guest rates dishes and service.	Feedback	INSERT.
Reports	Revenue, top dishes, peak hours.	vw_daily_revenue, Payment, OrderItem, Reservation	SELECT with GROUP BY.
Security and audit	Role permissions and change log.	Staff, AuditLog	GRANT; trigger inserts.

3.4 CRUD Operations

The application performs Create, Read, Update and Delete operations on all operational data. Records are not physically deleted during normal work: tickets, reservations and orders move through status values, so that the restaurant keeps a complete history. Table 12 lists the main operations.

Table 12. CRUD Operations

Operation	Application Function	Table(s) Affected	SQL / Evidence
Create	Register guest	Customer	INSERT INTO Customer (full_name, phone) VALUES (...)
Create	Reserve table	Reservation	CALL sp_reserve_table(...) inserts one row
Create	Place order	Orders, OrderItem	CALL sp_place_order(...) inserts one order and its dish rows
Create	Generate bill / record payment	Bill, Payment	CALL sp_generate_bill(...); INSERT INTO Payment (...)
Read	Free tables, today's reservations	DiningTable, Reservation	SELECT from vw_free_tables; SELECT with JOIN and CURDATE()
Read	Kitchen queue, reports	Orders, Payment	SELECT from vw_kitchen_queue and vw_daily_revenue
Update	Seat guest, change order status	Reservation, Orders	UPDATE Orders SET status = 'PREPARING' (trigger checks the sequence)
Update	Change dish price or availability	MenuItem	UPDATE MenuItem SET price = ...; AuditLog row created
Delete (logical)	Cancel a reservation	Reservation	UPDATE Reservation SET status = 'CANCELLED'
Delete (physical)	Remove an unused dish	MenuItem	DELETE allowed only for manager; blocked by foreign key if ordered before

3.5 Business Rules, Validation and Error Handling

Table 13 shows how business rules are enforced in layers: constraints, procedures, triggers and roles.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Valid reservation slot	CHECK (slot_end > slot_start)	Insert rejected by the database.
No overlapping reservation on a table	Overlap predicate in sp_reserve_table, row lock and UNIQUE (table_id, slot_start)	Another table is chosen, or NO TABLE FREE.
Party must fit the table	capacity >= party_size in the allocation query	Smallest fitting table assigned.
Positive dish quantity	CHECK (quantity > 0)	Order row rejected.
One line per dish in an order	UNIQUE (order_id, item_id)	Duplicate line rejected.
Only available dishes can be ordered	is_available = 1 filter in sp_place_order	Unavailable dish is skipped or refused.
Kitchen status sequence	BEFORE UPDATE trigger trg_orders_before_update uses SIGNAL SQLSTATE '45000'	'Invalid status change' error.
Stock cannot go negative	CHECK (stock_qty >= 0) on Ingredient and trigger deduction	Order refused when stock is insufficient.
Coupon must be valid	sp_generate_bill checks is_active and the validity dates	Discount is zero for an invalid coupon.
One payment per bill	UNIQUE (bill_id) on Payment	Second payment rejected.
Role restrictions	GRANT per role (Table 14 and Appendix A)	'Access denied' error.
Audit of sensitive changes	AFTER UPDATE and AFTER INSERT triggers on Bill, Payment and MenuItem	Row written to AuditLog.
Failure safety	EXIT HANDLER with ROLLBACK inside procedures	No partial reservation or order is saved.

3.6 Reports and Database Operations

Reporting is implemented through views and aggregate queries. Table 14 lists the reports, their SQL logic and the output the application shows.

Table 14. Reports and Database Operations

Report / Operation	SQL Logic	Application Output
Free tables	View vw_free_tables: tables with no current reservation	Table numbers with capacity.
Kitchen queue	View vw_kitchen_queue: open orders (PLACED or PREPARING) with their dishes	Orders waiting for the chef.
Daily revenue	View vw_daily_revenue: SUM(amount) from Payment GROUP BY DATE(paid_at)	Revenue per day.
Top-selling dishes	JOIN OrderItem and MenuItem, SUM(quantity), GROUP BY dish, ORDER BY total DESC	Ranked dish list and chart.
Peak reservation hours	COUNT of Reservation GROUP BY HOUR(slot_start)	Busiest booking hours.
Low-stock ingredients	WHERE stock_qty <= reorder_level	Ingredients to reorder.
Roles and permissions	GRANT statements for manager, waiter and chef	Each role sees only what it needs.
Concurrency control	START TRANSACTION, SELECT … FOR UPDATE, COMMIT	Second booking waits, then re-checks.

Indexes. Three indexes support the most frequent lookups: a composite index on Reservation (table_id, slot_start) makes the overlap check avoid full table scans, an index on Orders (status) speeds up the kitchen queue, and an index on MenuItem (category_id) speeds up menu display.

Concurrency. Two staff members may see the last table as free and both try to reserve it, which would create a double booking. The reservation therefore runs in one transaction (all or nothing, in line with ACID). SELECT … FOR UPDATE makes the second booking wait

until the first one commits; the second session then re-checks the overlap and is refused or moved to another table. The unique key on table and slot is the final safety net.